

Le polymorphisme

Le polymorphisme est un mécanisme important dans la programmation objet. Il permet de modifier le comportement d'une classe fille par rapport à sa classe mère. Le polymorphisme permet d'utiliser l'héritage comme un mécanisme d'extension en adaptant le comportement des objets.

Principe du polymorphisme

Prenons l'exemple de la classe *Animal*. Cette classe offre une méthode *crier*. Pour simplifier notre exemple, la méthode se contente d'écrire le cri de l'animal sur la sortie standard.

```
package fr.epsi.b3.animal;

public class Animal {

    public void crier() {
        System.out.println("un cri d'animal");
    }

}
```

Nous disposons également des classes *Chat* et *Chien* qui héritent de la classe *Animal*.

```
package fr.epsi.b3.animal;

public class Chat extends Animal {

}
```

```
package fr.epsi.b3.animal;

public class Chien extends Animal {
```



Ces deux classes sont une spécialisation de la classe *Animal*. À ce titre, elles peuvent **redéfinir** (*override*) la méthode *crier*.

```
package fr.epsi.b3.animal;

public class Chat extends Animal {

    public void crier() {
        System.out.println("Miaou !");
    }

}
```

```
package fr.epsi.b3.animal;

public class Chien extends Animal {

    public void crier() {
        System.out.println("Whouaf whouaf !");
    }

}
```

Chaque classe fille change le comportement de la méthode *crier*. Cela signifie qu'un objet de type *Chien* pour lequel on invoque le méthode *crier* ne fournira pas le même comportement qu'un objet de type *Chat*. Et cela, quel que soit le type de la variable qui référence ces objets.

```
Animal animal = new Animal();
animal.crier(); // affiche "un cri d'animal."

Chat chat = new Chat();
chat.crier(); // affiche "Miaou !"

Chien chien = new Chien();
chien.crier(); // affiche "Whouaf whouaf !"

animal = chat;
animal.crier(); // affiche "Miaou !"

animal = chien;
animal.crier(); // affiche "Whouaf whouaf !"
```

L'exemple de code ci-dessus montre que l'implémentation de la méthode *crier* dépend du type réel de l'objet et non pas du type de la variable qui le référence.



Une exception : les méthodes privées

Les méthodes de portée **private** ne supportent pas le polymorphisme. En effet, une méthode de portée **private** n'est accessible uniquement que par la classe qui la déclare. Donc si une classe mère et une classe fille définissent une méthode **private** avec la même signature, le compilateur les traitera comme deux méthodes différentes.

Redéfinition et signature de méthode

Le principe du polymorphisme repose en Java sur la redéfinition de méthodes. Pour que la redéfinition fonctionne, il faut que la méthode qui redéfinit possède une signature *correspondante* à celle de la méthode originale.

Le cas le plus simple est celui de l'exemple précédent. Les méthodes ont exactement la même signature : même portée, même type de retour, même nom et mêmes paramètres.

Cependant, la méthode qui redéfinit peut avoir une signature légèrement différente.

Une méthode qui redéfinit, peut avoir une portée différente si et seulement si, celle-ci est plus permissive que celle de la méthode d'origine. Il est donc possible d'augmenter la portée de la méthode mais jamais de la réduire :

- Une méthode de portée **package** peut être redéfinie avec la portée **package** mais aussi **protected** ou **public**.
- Une méthode de portée **protected** peut être redéfinie avec la portée **protected** ou **public**.
- Une méthode de portée **public** ne peut être redéfinie qu'avec la portée **public**.

Le changement de portée dans la redéfinition sert la plupart du temps à placer une implémentation dans la classe parente mais à laisser les classes filles qui le désirent offrir publiquement l'accès à cette méthode.

Au [chapitre précédent](#), nous avons introduit les classes *Vehicule*, *Voiture* et *Moto*. En partant du principe que seules les instances de *Voiture* peuvent offrir la méthode *reculer*, nous avons ajouté cette méthode dans la classe *Voiture*. Pour cela, nous avons dû modifier l'implémentation de la classe *Vehicule* en utilisant une portée **protected** pour l'attribut *vitesse*. Nous avons alors vu que cela n'était pas totalement conforme au [principe du ouvert/fermé](#).



```

package fr.epsi.b3.conduite;

public class Vehicule {

    private final String marque;
    protected float vitesse;

    public Vehicule(String marque) {
        this.marque = marque;
    }

    public void accelerer(float deltaVitesse) {
        this.vitesse += deltaVitesse;
    }

    public void decelerer(float deltaVitesse) {
        this.vitesse = Math.max(this.vitesse - deltaVitesse, 0f);
    }

    // ...
}

```

```

package fr.epsi.b3.conduite;

public class Voiture extends Vehicule {

    public Voiture(String marque) {
        super(marque);
    }

    public void reculer(float vitesse) {
        this.vitesse = -vitesse;
    }

    // ...
}

```



Nous pouvons maintenant revoir notre implémentation. En fait, c'est la méthode *reculer* qui doit être déclarée dans la classe *Véhicule* avec une portée **protected**. La classe *Voiture* peut se limiter à redéfinir cette méthode en la rendant **public**.



```

package fr.epsi.b3.conduite;

public class Vehicule {

    private final String marque;
    private float vitesse;

    public Vehicule(String marque) {
        this.marque = marque;
    }

    public void accelerer(float deltaVitesse) {
        this.vitesse += deltaVitesse;
    }

    public void decelerer(float deltaVitesse) {
        this.vitesse = Math.max(this.vitesse - deltaVitesse, 0f);
    }

    protected void reculer(float vitesse) {
        this.vitesse = -vitesse;
    }

    // ...
}

```

```

package fr.epsi.b3.conduite;

public class Voiture extends Vehicule {

    public Voiture(String marque) {
        super(marque);
    }

    public void reculer(float vitesse) {
        super.reculer(vitesse);
    }

    // ...
}

```

Dans l'exemple ci-dessus, le mot-clé **super** permet d'appeler l'implémentation de la méthode fournie par la classe *Vehicule*. Ainsi l'attribut *vitesse* peut rester de portée **private** et les classes filles de *Vehicule* peuvent ou non donner publiquement l'accès à la méthode *reculer*.

Une méthode qui redéfinit peut avoir un type de retour différent de la méthode d'origine à condition qu'il s'agisse d'une classe qui hérite du type de retour. Si nous reprenons l'exemple d'héritage des classes *Animal*, *Chien* et *Chat*, nous pouvons définir une classe *EleveurAnimal* qui permet de créer une instance de *Animal* quand on appelle la méthode *elever* :



```
package fr.epsi.b3.animal;

public class EleveurAnimal {

    public Animal elever() {
        return new Animal();
    }

}
```

Si maintenant nous créons la classe *EleveurChien* qui hérite de la classe *EleveurAnimale*, la redéfinition de la méthode *elever* peut déclarer qu'elle retourne un type *Chien*. Comme *Chien* hérite de *Animal*, la méthode qui redéfinit se présente comme une spécialisation.

```
package fr.epsi.b3.animal;

public class EleveurChien extends EleveurAnimal {

    public Chien elever() {
        return new Chien();
    }

}
```

Le mot-clé super

La redéfinition de méthode masque la méthode de la classe parente. Cependant, nous avons vu précédemment avec l'exemple de la méthode *reculer* que l'implémentation d'une classe fille a la possibilité d'appeler une méthode de la classe parente en utilisant le mot-clé **super**. L'appel à l'implémentation parente est très utile lorsque l'on veut effectuer une action avant et/ou après sans avoir besoin de dupliquer le code d'origine.



```
package fr.epsi.b3.conduite;

public class Voiture extends Vehicule {

    public Voiture(String marque) {
        super(marque);
    }

    public void accelerer(float deltaVitesse) {
        // faire quelque chose avant

        super.accelerer(deltaVitesse);

        // faire quelque chose après
    }

    // ...
}
```

Il existe tout de même une limitation : si une méthode a été redéfinie plusieurs fois dans l'arborescence d'héritage, le mot-clé `super` ne permet d'appeler que l'implémentation de la classe parente. Si la classe `Voiture` a redéfini la méthode `accelerer` et que l'on crée la classe `VoitureDeCourse` héritant de la classe `Voiture`.

```
package fr.epsi.b3.conduite;

public class VoitureDeCourse extends Voiture {

    public VoitureDeCourse(String marque) {
        super(marque);
    }

    public void accelerer(float deltaVitesse) {
        // faire quelque chose avant

        super.accelerer(deltaVitesse);

        // faire quelque chose après
    }

    // ...
}
```

La redéfinition de la méthode `accelerer` peut appeler l'implémentation de `Voiture` mais il est impossible d'appeler directement l'implémentation d'origine de la classe `Vehicule` depuis la classe `VoitureDeCourse`.



L'annotation @Override

Les annotations sont des types spéciaux en Java qui commence par @. Les annotations servent à ajouter une information sur une classe, un attribut, une méthode, un paramètre ou une variable. Une annotation apporte une information au moment de la compilation, du chargement de la classe dans la JVM ou lors de l'exécution du code. Le langage Java proprement dit utilise relativement peu les annotations. On trouve cependant l'annotation `@Override` qui est très utile pour le polymorphisme. Cette annotation s'ajoute au début de la signature d'une méthode pour préciser que cette méthode est une redéfinition d'une méthode héritée. Cela permet au compilateur de vérifier que la signature de la méthode correspond bien à une méthode d'une classe parente. Dans le cas contraire, la compilation échoue.

```
package fr.epsi.b3.conduite;

public class Voiture extends Vehicule {

    public Voiture(String marque) {
        super(marque);
    }

    @Override
    public void reculer(float vitesse) {
        super.reculer(vitesse);
    }

    // ...
}
```

Les méthodes de classe

Les méthodes de classe (déclarées avec le mot-clé **static**) ne supportent pas la redéfinition. Si une classe fille déclare une méthode **static** avec la même signature que dans la classe parente, ces méthodes seront simplement vues par le compilateur comme deux méthodes distinctes.

```
package fr.epsi.b3;

public class Parent {

    public static void methodeDeClasse() {
        System.out.println("appel à la méthode de la classe Parent");
    }
}
```




```
package fr.epsi.b3;

public class Enfant extends Parent {

    public static void methodeDeClasse() {
        System.out.println("appel à la méthode de la classe Enfant");
    }

}
```

Dans le code ci-dessus, la classe *Enfant* hérite de la classe *Parent* et toutes deux implémentent une méthode **static** appelée *methodeDeClasse*. Le code suivant peut être source d'incompréhension :

```
Parent a = new Enfant();
a.methodeDeClasse();

Enfant b = new Enfant();
b.methodeDeClasse();
```

Le résultat de l'exécution de ce code est :

```
appel à la méthode de la classe Parent
appel à la méthode de la classe Enfant
```

Comme les méthodes sont **static**, la redéfinition ne s'applique pas et la méthode appelée dépend du type de la variable et non du type de l'objet référencé par la variable. Cet exemple illustre pourquoi il est très fortement conseillé d'appeler les méthodes **static** à partir du nom de la classe et non pas d'une variable afin d'éviter toute ambiguïté.

```
Parent.methodeDeClasse();
Enfant.methodeDeClasse();
```

Méthode finale

Une méthode peut avoir le mot-clé **final** dans sa signature. Cela signifie que cette méthode ne peut plus être redéfinie par les classes qui en hériteront. Tenter de redéfinir une méthode déclarée **final** conduit à une erreur de compilation. L'utilisation du mot-clé **final** pour une méthode est réservée à des cas très spécifiques (et très rares). Par exemple si on souhaite s'assurer qu'une méthode aura toujours le même comportement même dans les classes qui en



Même si les méthodes **static** n'autorisent pas la redéfinition, elles peuvent être déclarées **final**. Dans ce cas, il n'est pas possible d'ajouter une méthode de classe qui a la même signature dans les classes qui en héritent.

Constructeur et polymorphisme

Les constructeurs sont des méthodes particulières qu'il n'est pas possible de redéfinir. Les constructeurs créent une séquence d'appel qui garantit qu'ils seront exécutés en commençant par la classe la plus haute dans la hiérarchie d'héritage. Puisque toutes les classes Java héritent de la classe **Object**, cela signifie que le constructeur de **Object** est toujours appelé en premier.

Cependant un constructeur peut appeler une méthode et dans ce cas le polymorphisme s'applique. Comme les constructeurs sont appelés dans l'ordre de la hiérarchie d'héritage, cela signifie qu'un constructeur invoque toujours une méthode redéfinie avant que la classe fille qui l'implémente n'ait pu être initialisée.

Par exemple, si nous disposons d'une classe *VehiculeMotorise* qui redéfinit la méthode *accelerer* pour prendre en compte la consommation d'essence :

```
package fr.epsi.b3.conduite;

public class VehiculeMotorise extends Vehicule {

    private Moteur moteur;

    public VehiculeMotorise(String marque) {
        super(marque);
        this.moteur = new Moteur();
    }

    @Override
    public void accelerer(float deltaVitesse) {
        moteur.consommer(deltaVitesse);
        super.accelerer(deltaVitesse);
    }

    // ...
}
```

Si maintenant nous faisons évoluer la classe *Vehicule* pour créer une instance avec une vitesse minimale :



```
package fr.epsi.b3.conduite;

public class Vehicule {

    private final String marque;
    protected float vitesse;

    public Vehicule(String marque) {
        this.marque = marque;
        this.accelerer(10);
    }

    public void accelere(float deltaVitesse) {
        this.vitesse += deltaVitesse;
    }

    // ...
}
```

Que va-t-il se passer à l'exécution de ce code :

```
VehiculeMotorise vehiculeMotorise = new VehiculeMotorise("DeLorean");
```

Le constructeur de *VehiculeMotorise* commence par appeler le constructeur de *Vehicule*. Ce dernier appelle implicitement le constructeur de *Object* (qui ne fait rien) puis il initialise l'attribut *marque* et il appelle la méthode *accelerer*. Comme cette dernière est redéfinie, c'est en fait l'implémentation fournie par *VehiculeMotorise* qui est appelée. Cette implémentation commence par appeler une méthode sur l'attribut *moteur* qui n'a pas encore été initialisé. Donc sa valeur est nulle et donc la création d'une instance de *VehiculeMotorise* échoue systématiquement avec une erreur du type *NullPointerException*.

On voit par cet exemple que l'appel de méthode dans un constructeur peut amener à des situations complexes. Il est fortement recommandé d'appeler dans un constructeur des méthodes dont le comportement ne peut pas être modifié par la redéfinition : soit des méthodes déclarées **private** soit des méthodes déclarées **final**.

Masquage des attributs par héritage

Il est possible de déclarer dans une classe fille un attribut portant le même nom que dans la classe parente. Cependant ceci ne correspond ni à une redéfinition ni au principe du polymorphisme. L'attribut de la classe fille se contente de masquer l'attribut de la classe parente.

Si l'attribut est de portée **private**, créer un attribut avec le même nom dans une classe fille n'a aucun impact particulier. Cela permet d'isoler l'état interne de la classe parente par rapport à sa classe fille.



Si l'attribut est de portée package, **protected** ou **public** alors l'attribut de la classe parente est simplement masqué dans la classe fille. Si une classe fille veut accéder à l'attribut de la classe parente, elle peut le faire à travers le mot-clé **super**.

```
package fr.epsi.b3;

public class Personne {

    protected String nom;

    public Personne(String nom) {
        this.nom = nom;
    }

    // ...

}
```

```
package fr.epsi.b3;

public class HerosMasque extends Personne {

    private String nom;

    public HerosMasque(String nom, String nomHeros) {
        super(nom);
        this.nom = nomHeros;
    }

    @Override
    public String toString() {
        // mmmh ! Pas très clair
        return this.nom + " dont l'identité secrète est " + super.nom;
    }

    // ...

}
```

En raison de la difficulté de compréhension que cela peut entraîner, il est préférable de ne jamais créer dans une classe fille un attribut portant le même nom que celui d'un attribut de portée package, **protected** ou **public** d'une de ses classes parentes.

Le principe du ouvert/fermé

Le principe du ouvert/fermé stipule qu'une classe doit être conçue pour être ouverte en extension mais fermée en modification.



D'un côté, si une classe hérite d'une autre classe, elle doit pouvoir ajouter des nouveaux comportements avec de nouvelles méthodes. Par contre la redéfinition de méthode ne doit pas être utilisée pour créer une implémentation qui a un comportement trop différent de celui de la classe parente.

D'une autre côté, si une classe hérite d'une autre classe, elle ne doit pas pouvoir modifier le fonctionnement décrit par la classe parente. En Java, pour interdire de modifier le comportement d'une classe, on peut déclarer ses attributs **private** et les méthodes jugées les plus critiques peuvent être déclarées **final**.

